

Programming of ARM7 Peripherals using C

6.1 ARM7 SoC

- ARM ICs are available, manufactured by companies such as NXP, ST Microelectronics, Atmel etc
- Currently all have switched to the latest Cortex Series
- ARM7 is still widely used processor
- We choose SoC of NXP which is popular in the industry
- NXP ARM7 portfolio includes the LPC2100, LPC2200, LPC2300, LPC2400
- We will choose LPC214x series which have made their venture in many important applications
- This series consists of LPC2141, LPC2142, LPC2146, LPC2144 and LPC2148
- The Keil IDE has 'simulation' capabilities and we can simulate the working of few peripherals available.

Features of LPC 214x series

- 16-bit/32-bit ARM7TDMI-S microcontroller in a tiny LQFP64 package
- 8 kB to 40 kB of on-chip static RAM and 32kB-512 kB of on-chip flash memory
- 128-bit wide interface/accelerator enables high-speed 60 MHz operation
- In-system programming/Application Programming via on-chip boot loader software. Single flash sector or full chip erase in ms. The LPC2146/2148 provide 8kb of on chip RAM accessible to USB by DMA.
- Embedded ICE RT and Embedded Trace interfaces offer real-time debugging with the on-chip Real Monitor software and high-speed tracing of instructions execution.
- USB 2.0 Full-speed compliant device controller with 2kB of endpoint RAM
- One or two 10-bit ADCs provide a total of 6/14 analog inputs with conversion times as slow 2.44micro seconds per channel.

Features of LPC 214x series contd...

- Single 10-bit DAC provides variable analog output.
- Two 32-bit timers/external event counters, PWM unit and watchdog.
- Low power Real-Time Clock (RTC) with independent power and 32 kHz clock input.
- Multiple serial interfaces including two UARTs, two Fast 12C bus, SPI and SSP with buffering and variable data length capabilities
- Vectored Interrupt Controller (VIC) with configurable priorities and vector addresses.
- Up to 45 of 5 V tolerant fast general purpose I/O pins in a tiny LQFP64 package
- Up to 21 external interrupt pins available

Features of LPC 214x series contd...

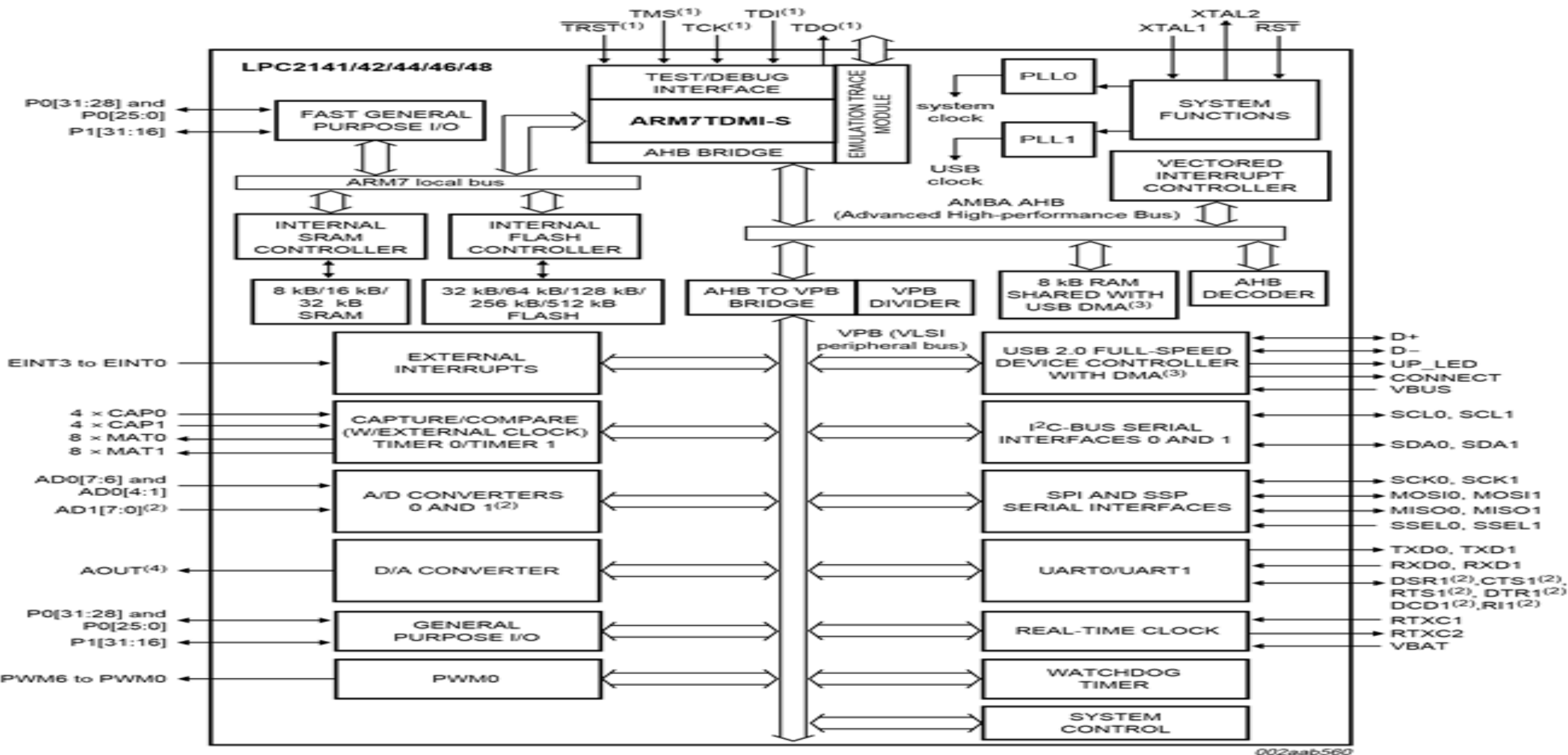
- 60 MHz maximum CPU clock available from programmable on-chip PLL
- On-chip integrated oscillator operates with an external crystal from 1 MHz to 25 MHz
- Power saving modes include Idle and Power-down
- Individual enable/disable of peripheral functions as well as peripheral clock scaling
- Processor wake-up from Power-down mode via external interrupt of BOD
- Single power supply chip with Power On Reset and Brown Out detector circuits

Important Features

1. 16-bit/32-bit ARM7TDMI-S microcontroller is the core.
2. 8KB-40KB of on chip static RAM and 32 KB to 512 KB of on-chip flash memory
3. One/two 10-bit ADCs with 6/14 analog input pins
4. 10-bit DAC
5. Timers – two 32-bit timers/external counters
6. PWM unit, with 6 output pins and a Watch dog timer

Important Features contd...

7. Real-Time Clock (RTC) with independent power and 32 kHz clock input
8. Serial Communication – Two UARTs, two SPI and Synchronous Serial Port and two I2C
9. Vectored Interrupt Controller with configurable priorities
10. The external crystal can have frequencies from 1MHz to 25 MHz
11. Low power modes for power saving
12. Power on Reset and Brown out Reset circuits
13. Two PLLs



(1) Pins shared with GPIO.

(2) LPC2144/46/48 only.

(3) USB DMA controller with 8 kB of RAM accessible as general purpose RAM and/or DMA is available in LPC2146/48 only.

(4) LPC2142/44/46/48 only.

Few peripherals

- Processor
 - Processor used in this chip family is ARM7TDMI-S with ARM and THUMB instruction capabilities.
- Memory map
 - The processor core has 32 address lines (internal to the chip)
 - Memory addressing capability is $2^{32} = 4$ GB.
 - All peripherals are also mapped to this memory since memory mapped I/O technique
- Flash memory
 - The amount of flash memory is not the same for all members of the family
 - Accordingly LPC2141/ 2/4/8 incorporates a 32kb,64kb,128kb,256kb and 512 kb flash memory system respectively.
 - This memory may be used for code and data storage. It starts from 0x00000000
- On-chip static RAM
 - LPC214x provides 8/16/32 kB of static RAM. It starts from 0x40000000
- System control block
 - Many functions are not related to any specific peripheral, but are common to the chip as a whole
 - Functions: Crystal oscillator, VPD divider, External Interrupt Inputs, PLL, Power Control, Reset, Wakeup Timer

Memory map of LPC214x

4.0 GB	AHB PERIPHERALS	0xFFFF FFFF
3.75 GB	VPB PERIPHERALS	0xF000 0000
3.5 GB		0xE000 0000
3.0 GB	RESERVED ADDRESS SPACE	0xC000 0000
2.0 GB	BOOT BLOCK (12 KB REMAPPED FROM ON-CHIP FLASH MEMORY)	0xB000 0000 0x7FFF FFFF
	RESERVED ADDRESS SPACE	0x7FFF D000 0x7FFF CFFF
	8 KB ON-CHIP USB DMA RAM (LPC2146/2148)	0x7FD0 2000 0x7FD0 1FFF
	RESERVED ADDRESS SPACE	0x7FD0 0000 0x7FCF FFFF
	32 KB ON-CHIP STATIC RAM (LPC2146/2148)	0x4000 8000 0x4000 7FFF
	16 KB ON-CHIP STATIC RAM (LPC2142/2144)	0x4000 4000 0x4000 3FFF
	8 KB ON-CHIP STATIC RAM (LPC2141)	0x4000 2000 0x4000 1FFF
1.0 GB	RESERVED ADDRESS SPACE	0x4000 0000 0x3FFF FFFF
	TOTAL OF 512 KB ON-CHIP NON-VOLATILE MEMORY (LPC2148)	0x0008 0000 0x0007 FFFF
	TOTAL OF 256 KB ON-CHIP NON-VOLATILE MEMORY (LPC2146)	0x0004 0000 0x0003 FFFF
	TOTAL OF 128 KB ON-CHIP NON-VOLATILE MEMORY (LPC2144)	0x0002 0000 0x0001 FFFF
	TOTAL OF 64 KB ON-CHIP NON-VOLATILE MEMORY (LPC2142)	0x0001 0000 0x0000 FFFF
	TOTAL OF 32 KB ON-CHIP NON-VOLATILE MEMORY (LPC2141)	0x0000 8000 0x0000 7FFF
0.0 GB		0x0000 0000

Few
peripherals

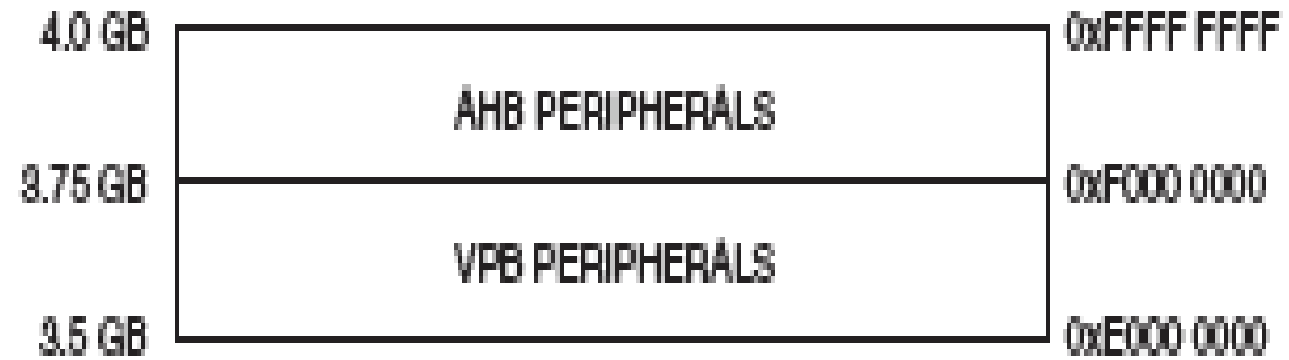


Figure 6.3 Address
map of peripherals

AHB –Advanced High Speed Peripheral Bus
VPB –VLSI peripheral Bus

Few peripherals

- System control block
 - Many functions are not related to any specific peripheral, but are common to the chip as a whole
 - Functions: Crystal oscillator, VPB divider, External Interrupt Inputs, Phase locked loop(PLL), Power Control, Reset, Wakeup Timer

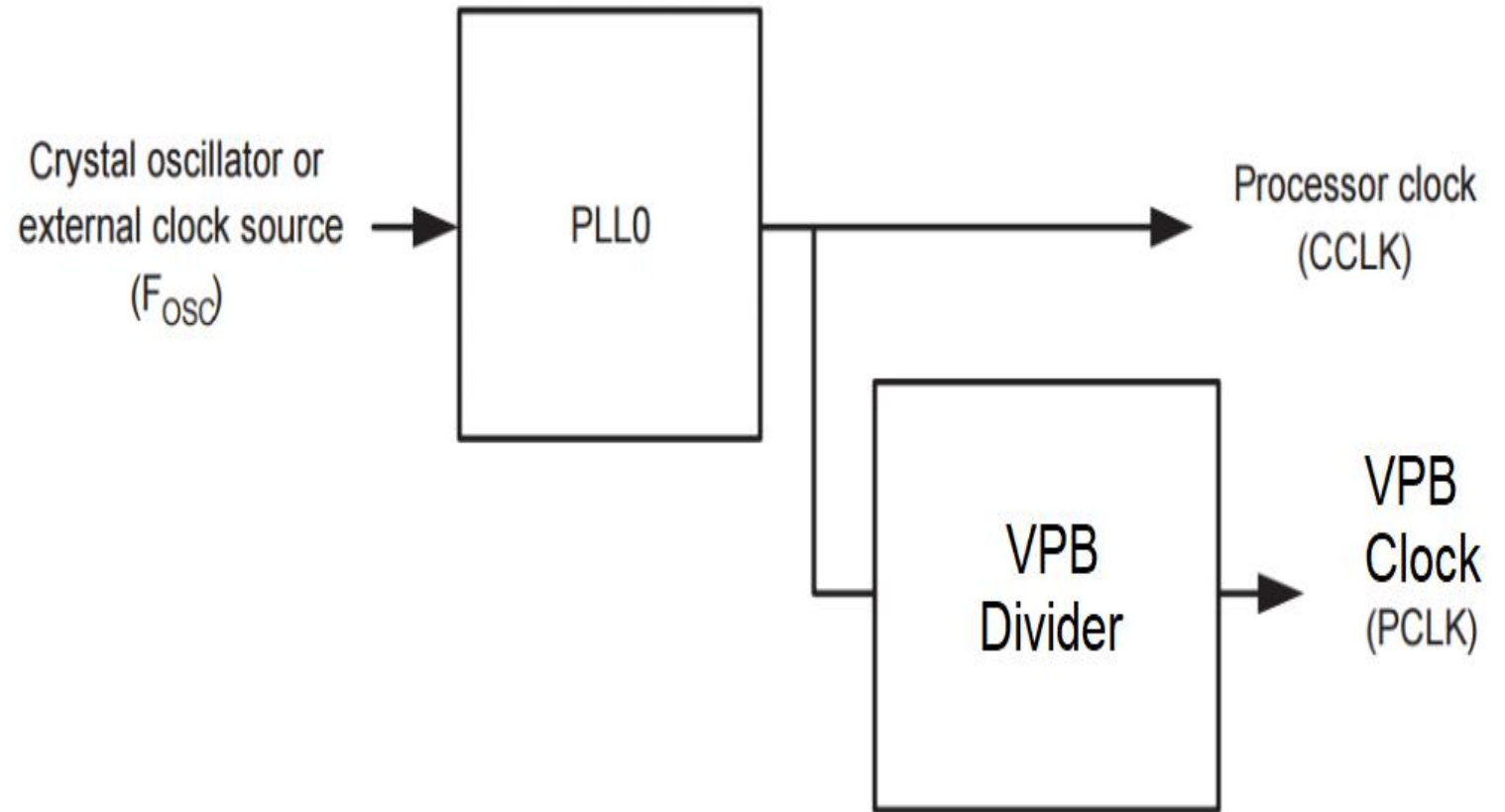
Crystal oscillator and PLL

- There are various options for the oscillator which supplies the system clock.
- External frequency by connecting an external oscillator, supplying a 50% duty cycle signal in the range of 1-50 MHz
- There is also an on-board oscillator and for that an external crystal with frequency in the range of 1-30 MHz is allowed.
- There are two PLLs and one is used entirely for the generation of the system clock, noted as CCLK.
- The PLL accepts an input clock frequency in the range of 10-25MHz, which is multiplied to get a range of 10-60 MHz for the core clock
- The core clock is used entirely for generation of system clock noted as CCLK.
- The PLL accepts an input clock frequency in the range of 10MHz to 25MHz only, which is multiplied to get a range of 10-60MHz for the core clock.

VPB divider

- Volume Parameter Block(VPB)
- Usually, peripherals do not work at the high frequency that the core operates at.
- So the core clock is generally slowed down to one half or one fourth for use by peripherals.
- The default condition at reset is for the VPB bus to run at one quarter speed.
- The VPB divider determines the relationship between the processor core clock CCLK and the peripheral clock PCLK.
- Another reason to do the frequency division is to reduce power dissipation.

VPB divider connection



Low power modes

- The LPC214x supports two low power modes
 - Idle mode
 - Power-down mode.
- In idle mode, instruction execution is suspended until either a Reset or interrupt occurs.
- Peripherals can continue to operate during Idle mode and may generate interrupts to cause the processor to resume execution.
- This mode eliminates power used by the processor itself, memory systems and related controllers and internal buses.
- In power-down mode, the oscillator is shut down and the chip receives no internal clocks.
- The processor state and registers, peripheral registers and internal SRAM values are preserved throughout this mode and the logic levels of chip pins remain unchanged.
- This state can be exited and normal operation can be resumed by Reset or interrupt.
- Chip power consumption is nearly zero.

Wake up timer

- If the processor has gone to any of the low power modes, it will be awakened by a stimuli like an interrupt or reset signal.
- When wake up occurs, code execution for the required application should start.
- The function of the wakeup timer is to monitor the crystal oscillator to ensure that it is safe to begin code execution
- Once a clock is detected wakeup timer counts 4096 clocks.
- When the onboard modules initialization is complete, the processor is allowed to execute instructions.
- The function of the wakeup timer is to produce a standard amount of delay.

RESET

- There are two sources for Reset.
- One is the reset pin and other is the watchdog reset
- The RESET pin is a Schmitt Trigger input
- Reset occurs by one of these sources, the first action is of the starting of the wakeup timer
- The process of resetting continues until the on-chip circuitry is initialized and a fixed number of clocks have occurred after the oscillator starts running.

Reset sources and wake up timer

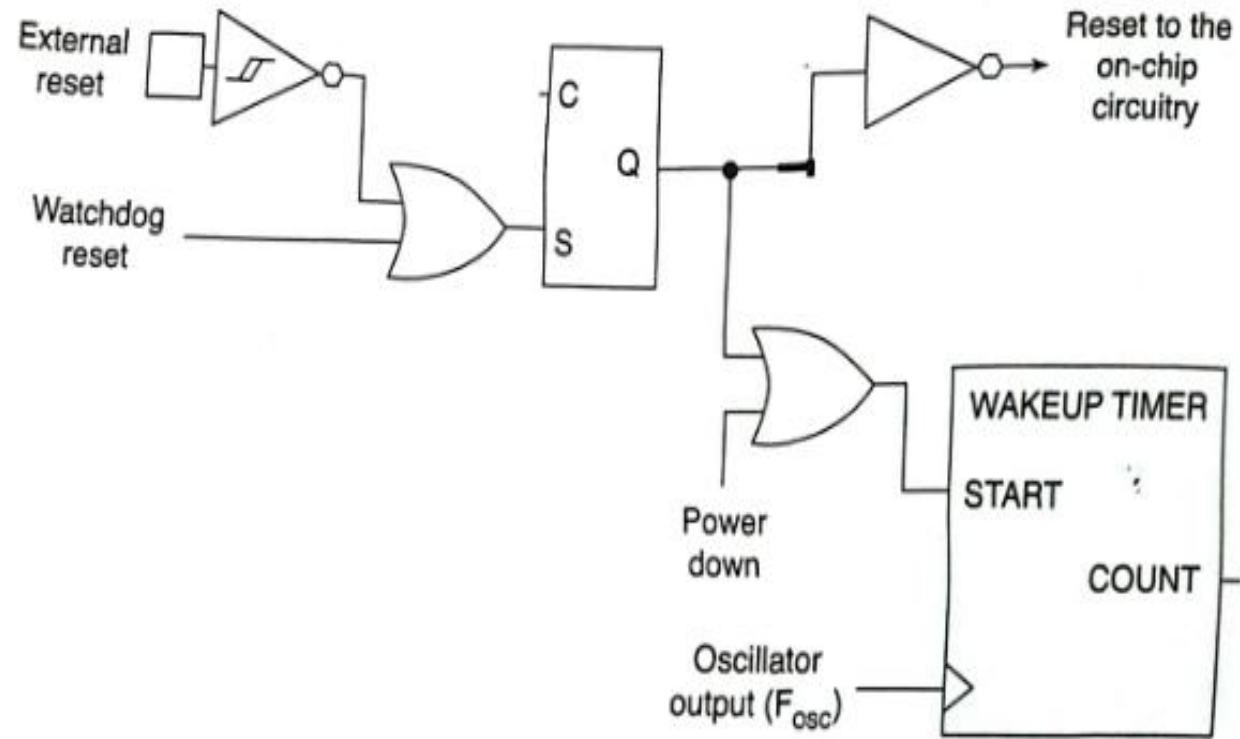


Figure 6.5 Relationship between reset sources and wake up timer

What does the glitch filter do?

- Removes unwanted high or low pulses from an input pin.
- Reset glitch filter allows the processor to ignore external reset pulses that are very short
- There should be minimum duration for the RESET pulse.
- When the internal reset is removed, the processor begins executing at address 0x0

Memory mapping control

Pins	Function	Bits of PINSELo register	Values
Port 0.0	GPIO	1:0	00
Port 0.1	RxD of UART0	3:2	01
Port 0.2	Capture	5:4	10
Port 0.14	SDA	29:28	11

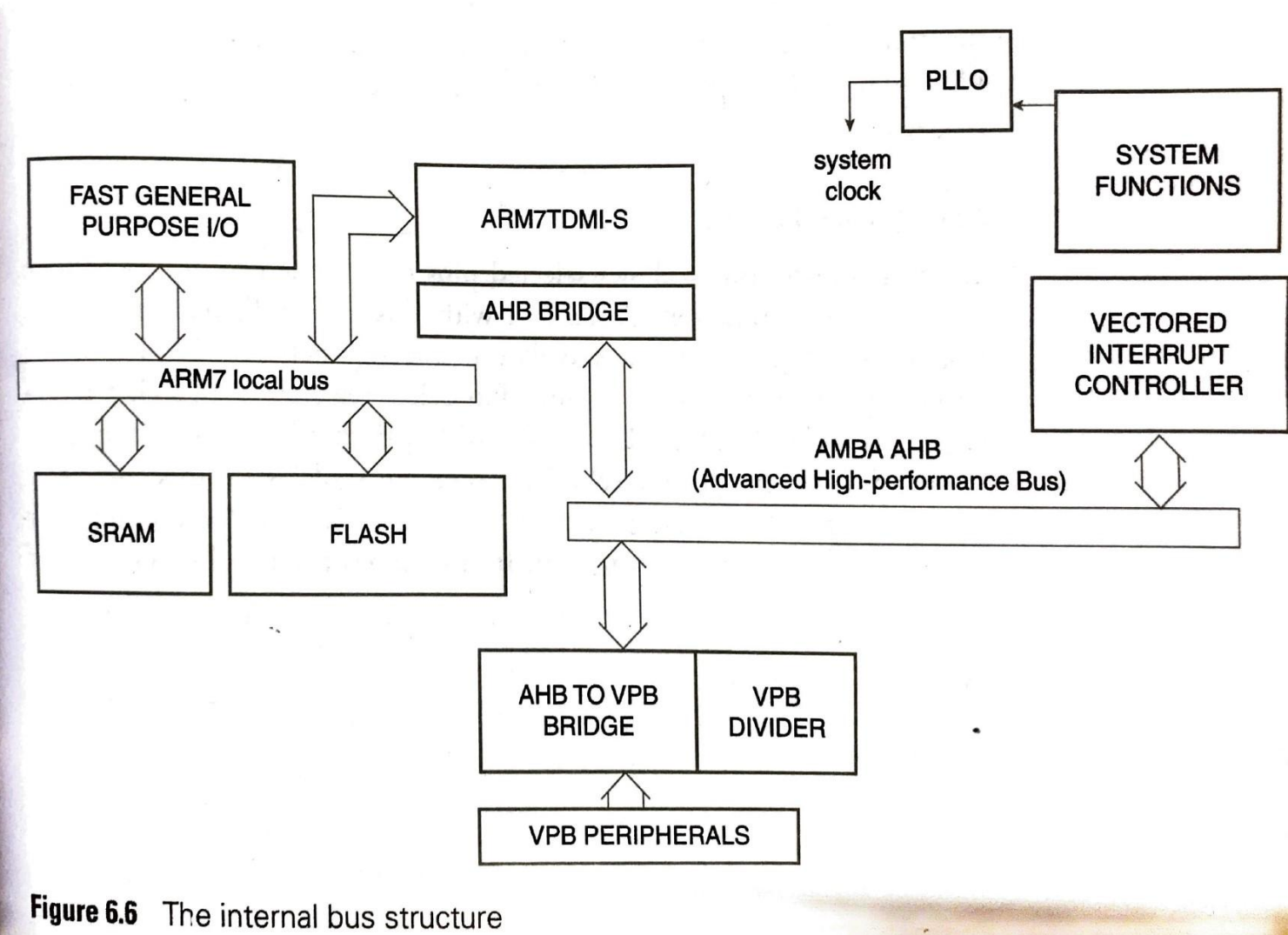
Memory mapping control contd...

- The interrupt vectors are all at the lower end of the memory map, beginning at address 0x00000000.
- There is not enough space in that region to write the ISR of each of the interrupts
- These vectors may be mapped to other parts of the flash memory, or to the on-chip RAM

Internal Bus

- Fig 6.6 shows the internal bus structure of the LPC214X family.
- The fastest bus which is the ARM local bus and it connects the ARM core to memory.
- Any high-speed GPIO may also use this bus.
- The system functions and the VIC (Vectored Interrupt Controller) are on AHB bus, while the peripherals use the low-speed VPB bus.
- Bridges are interfacing circuitry which convert one bus standard to another.
- Thus there is an AHB bridge which acts as an intermediary between the highest speed local bus and the AHB.
- The other bridge is the AHB to VPB bridge.

Internal bus



Pheripherals

The Peripherals:

- Initially, will understand the method of operation of each peripheral.
- Then simulate their working using keil simulator

General purpose input output

- The LPC 214x chips have two I/O ports designated as Port 0 and Port 1.
- Both are 32-bit I/O ports, but each pin of them can be configured and used individually as an input or output pin, except Po.31 which is an output only pin
- Pins Po.24, Po.26, Po.27 are not available.
- In Port1, pins P1.0 through 15 are not available.
- P1.16 to P1.25 are reserved.

Pin select block

- The pin connect block allows selected pins of the microcontroller to have more than one function.
- In LPC chip, each pin has four functions, selectable by the bits in pin select register.
- The pin connect block allows selected pins of the microcontroller to have more than one function.
- There are three Pin Select registers in this chip designated as PINSEL₀, PINSEL₁, PINSEL₂.
- Each of these 32 bit registers can handle 16 pins .

Four functions of Po.19

PinSelect 1 Reg bits	Port Pin no	Value	Pin function
7:6	Po.19	00	GPIO Port 0.19
		01	Match 1.2 (Timer 1)
		10	MOSI ₁ (SSP)
		11	Capture 1.2 (Timer 1)

Relevant bits of PINSELo

Pins	Functions	Bits of PINSELo	Values
Port 0.0	GPIO	1:0	00
Port 0.1	RxD of UART0	3:2	01
Port 0.2	Timero Capture	5:4	10
Port 0.14	SDA(I2C pin)	29:28	11

Using GPIO pins

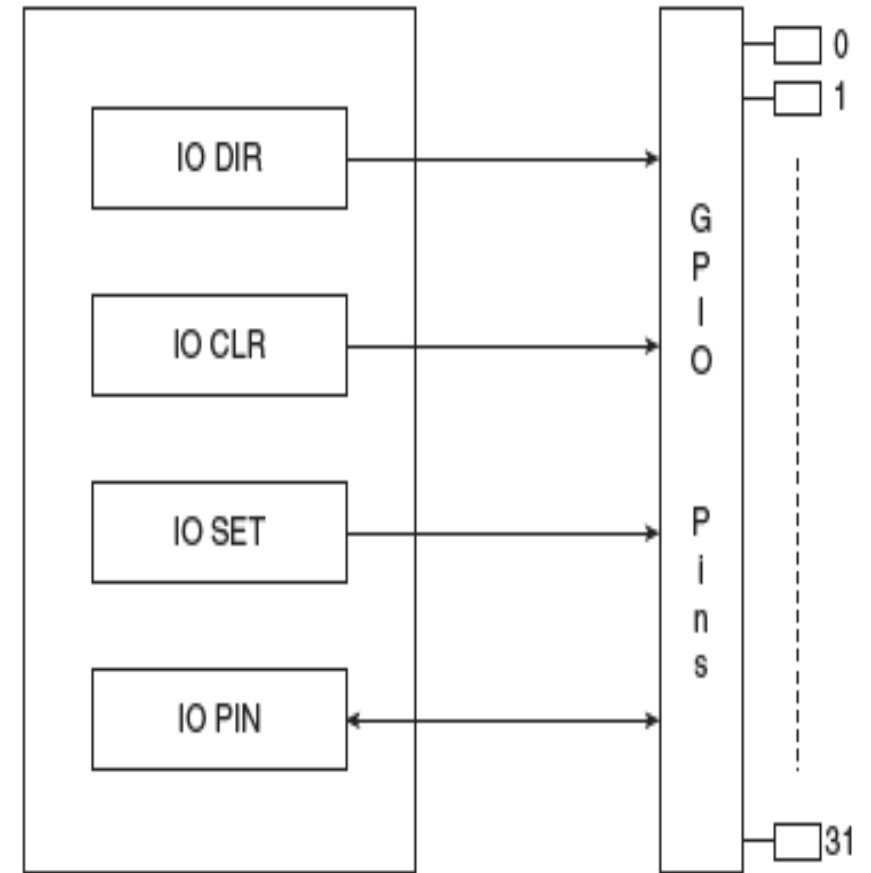
- Refer figure 6.7 from text book, which shows the conceptual view of chip with port1 I/O pins.
- As mentioned earlier that, pins 0-15 of port 1 are not available, pins p1.16 to p1.25 are reserved. Compared to port 1, for port 0, more number of pins are available for use as I/O
- Each pin can be used for four different functions ,but the default functionality is as a GPIO pin.
- It can be addressed individually and has the following attributes associated with it:
 - It may be an input or output pin
 - If it is an input pin, it reads in value (0 or 1) from outside the chip.
 - If it is an output pin, it may be set or cleared by software.

Using GPIO pins

- There are four registers associates with each GPIO
 - A direction register (IODIR) in which we can decide whether it is to be an output or input pin
 - If the pin is an output pin, there is a 'Set' register (IOSET) in which we can make a pin to have a logic level of '1' on it.
 - If the pin is an output pin, a 'clear' register (IOCLR) is available in which we can make a pin to have a logic level of '0' on it.
 - A 'PIN' register (IOPIN) register in which we can read the actual logic levels on the pin.
 - We can use this register for doing set/clear functions for the output port, but not on individual pins.

Functions of GPIO registers

- IOPIN: GPIO Port Pin value register
- IOSET: GPIO Port Output Set register
- IODIR: GPIO Port Direction control register
- IOCLR: GPIO Port Output Clear register



Functions of GPIO registers

Table 6.3 Functions of GPIO registers

Generic name	Description
IOPIN	GPIO Port Pin value register. The current state of the GPIO configured port pins can always be read from this register, regardless of pin direction.
IOSET	GPIO Port Output Set register. This register controls the state of output pins in conjunction with the IOCLR register. Writing ones produces highs at the corresponding port pins. Writing zeroes has no effect.
IODIR	GPIO Port Direction control register. This register individually controls the direction of each port pin.
IOCLR	GPIO Port Output Clear register. This register controls the state of output pins. Writing ones produces lows at the corresponding port pins and clears the corresponding bits in the IOSET register. Writing zeroes has no effect.

IOPIN register

- Port pins that are configured to perform only digital functions.
- It will give the logic value of the pin irrespective of the pin whether the pin is an input or output, and as GPIO or an alternate digital function.
- If a pin has an analog function as one of its options , the pin state cannot be read if the analog configuration is selected.
- Selecting the pin as an analog input disconnects the digital features of the pin .In that case , the pin value read in the IOPIN register is not valid.
- It is possible to write to the IOPIN register and then it stores the value in the port output register, bypassing the need to use both IOSET and IOCLR registers to obtain entire written value.
- This feature should be used carefully in an application since it affects the entire port.
- The IOPIN register may be used to write a value in the IO port register. But it cannot be used for toggling individual pin outputs.

Functions of GPIO registers

Write values to the Direction, Set and Clear registers, as specified below.

1. Make the upper 16 bits of Port 1 to be output pins.
2. Set the following bits P1.16 to P1.23.
3. Clear the bits P1.16 to P1.19.
4. What will be the value in the upper 16 bits of the IOPIN1 register after the above operations?

Solution:

The registers for Port1 are designated as IODIR1, IOSET1, IOCLR1.

1. To make the upper 16 bits to be output pins, the logic in the IODIR register is '1' for each bit.
Hence $\text{IODIR1} = 1111\ 1111\ 1111\ 1111\ 0000\ 0000\ 0000\ 0000 = 0xFFFF0000$.
2. To set, $\text{IOSET1} = 0000\ 0000\ 1111\ 1111\ 0000\ 0000\ 0000\ 0000 = 0x00FF0000$.
3. To clear, $\text{IOCLR1} = 0000\ 0000\ 0000\ 1111\ 0000\ 0000\ 0000\ 0000 = 0x000F0000$.
4. Because of the setting and clearing of specific pins as discussed above, the upper 16 bits of IOPIN1 will read as 0x00F0.

Functions of GPIO registers

Write a program to toggle the lowest pin of Port 0 with a delay between the two states.

```
#include<LPC214X.h>
void delay(int);

int main()
{
    IODIR0 = 0x00000001;
    while(1)
    {
        IOSET0 = 0x00000001;    //set bit P0.0
        delay(500);
        IOCLR0 = 0x00000001;    //clear bit P0.0
        delay(500);
    }
}

void delay(int n)
{
    int i=0;
    for(i = 0;i<n;i++);
}
```

Output wave forms at Po.o

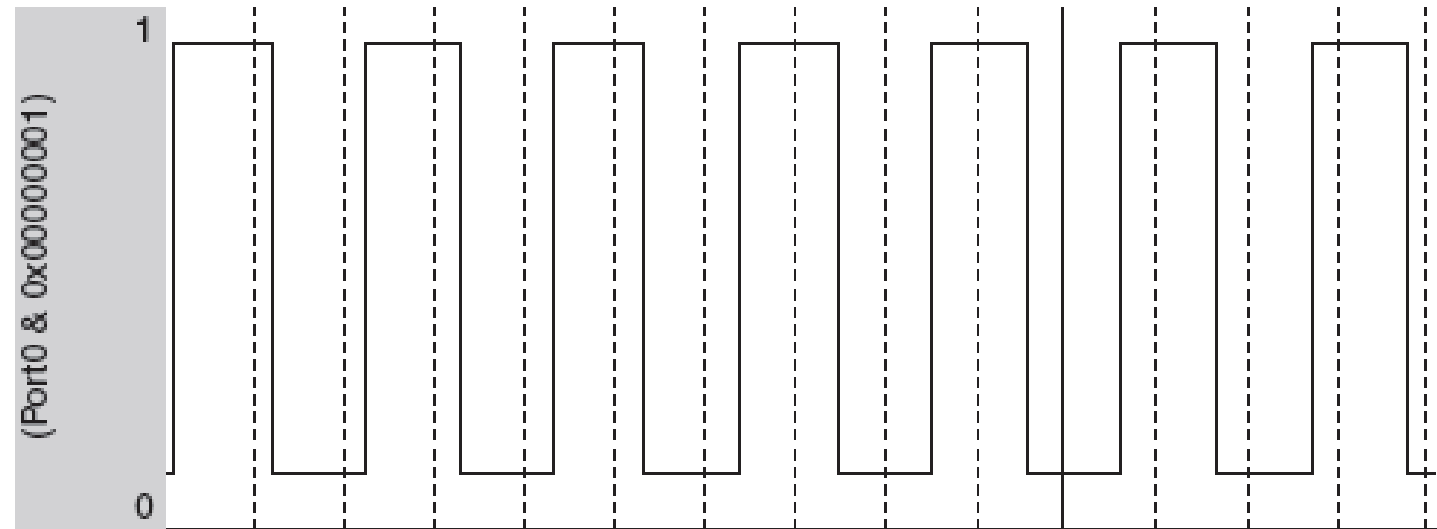


Figure 6.9 Output waveform at P0.0

Output wave forms at P1.28

```
#include<LPC214X.h>
void delay(int);
int main()
{
    IODIR1 = 0x10000000;
    while(1)
    {
        IOSET1 = 0x10000000;
        delay(500);
        IOCLR1 = 0x10000000;
        delay(1500);
        IOSET1 = 0x10000000;
        delay(1000);
        IOCLR1 = 0x10000000;
        delay(1000);
        IOSET1 = 0x10000000;
        delay(1500);
        IOCLR1 = 0x10000000;
        delay(500);
    }
}

void delay(int n){
    int i = 0;
    for(i = 0; i < n; i++);
}
```

Output wave forms at P1.28

It generates a waveform on pin Port1.28. The time period has different values as is shown in Fig. 6.10 which is the waveform obtained using the Logic Analyser in the Keil simulator.

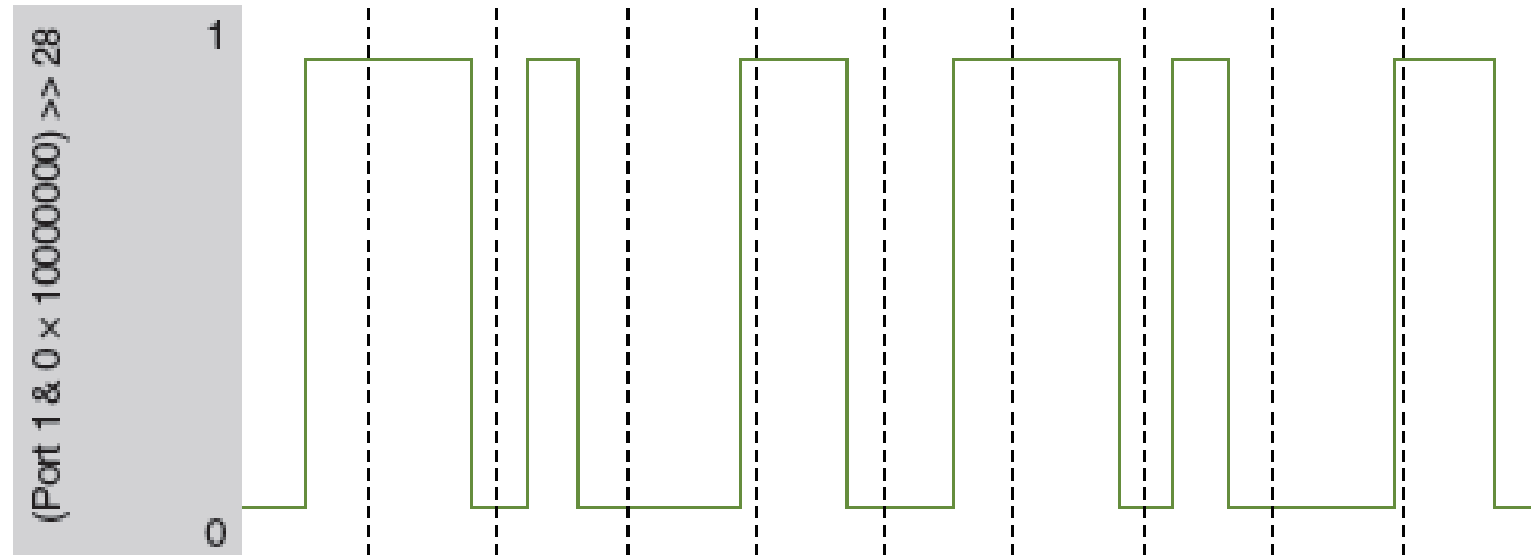


Figure 6.10 Waveform at P1.28

6.1.5.2 Timers

- The LPC214x family has two Timer/Counter units – Timer 0 and Timer 1, both of which are identical in features.
- It is designed to count cycles of the peripheral clock (PCLK) or an externally-supplied clock.
- In the PCLK unit is timer and in external clock unit is counter.
- The timer module can optionally generate interrupts or perform other actions at specified time values, based on four match registers.
- It also includes four capture inputs to trap the timer value when an input signal transitions, optionally generating an interrupt.

Operation as an interval timer

- Each timer is 32-bit Timer/Counter with a programmable 32-bit Prescaler.
- The steps are as follows:
 - A number is loaded into a 'Timer Count' register (TC)
 - There is a match register into which another number is loaded
 - The Timer Count Register is given a Count Enable (CE) signal and then it starts counting up, once per cycle of PCLK
 - The TC register value increases, and when it becomes equal to the number in the match register, we say that a 'match' occurs.
 - The event that follows the 'match' can be predecided by bit settings in the match control register(MCR).
 - The possibilities are
 - a. Stop Timer: Generate interrupt if operating in the interrupt mode
 - b. Reset timer count register Generate interrupt if operating in the interrupt mode
 - c. Continuous operation with optional interrupt generation on match

Registers for using the timer in the polled mode

Table 6.4 Registers for using the timer in the polled mode

Register	Description
TCR	Timer Control Register: The TCR is used to control the Timer Counter functions (enable/disable/reset).
TC	Timer Counter: The 32-bit TC is incremented every PR+1 cycles of PCLK. The TC is controlled through the TCR.
PR	Prescaler Register: This is used to specify the Prescaler value for incrementing the TC.
MCR	Match Control Register: The MCR is used to control the resetting of TC and generating of interrupt whenever a Match occurs.
MR0-MR3	Match Registers: The Match register values are continuously compared to the Timer Counter value. When the two values are equal, actions can be triggered automatically. The action possibilities are to generate an interrupt, reset the Timer Counter, or stop the timer. Actions are controlled by the settings in the MCR register.

Operation as an interval timer

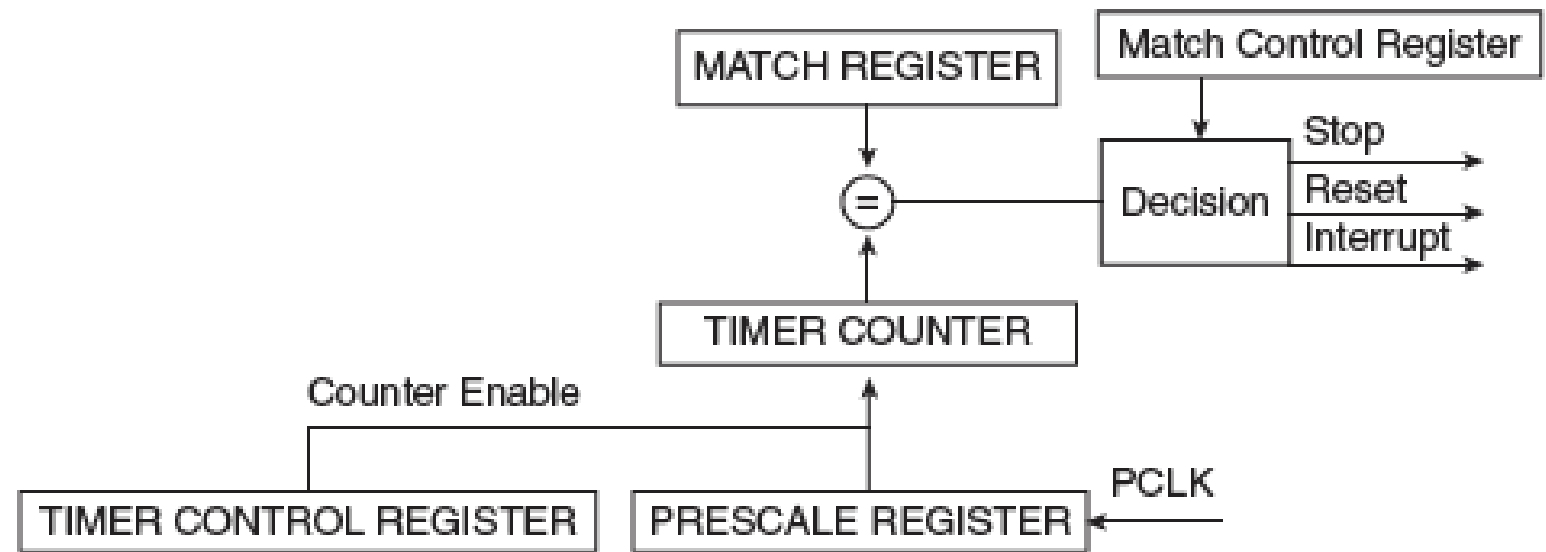


Figure 6.11 The block diagram showing steps in timer operation to create a delay

6.1.5.2.2 Registers for timing operation

There are two timers Timer0 and Timer1. For each timer, there are separate set of registers which are identified by adding 0 or 1 to the register name.

- **TIMER COUNT (TC) REGISTER:** This register increments by 1 for every PCLK or every pre-scaled PCLK.
- **TIMER CONTROL REGISTER (TCR):** This register controls the above mentioned Timer Count Register.
- There are two such registers, corresponding to Timer0 and Timer1 i.e ToCR and T1CR.
- TC can be enabled, disabled and reset using the TCR.
- Setting bit0, enables counting i.e count enable signal to start counting
- Bit1 is reset bit and when it is made 1 the Timer Count Register is reset at the next cycle of PCLK



Figure 6.12 Important bits of Timer Control Register

Registers for timing operation

- MATCH REGISTERS – MR₀ to MR₃: There are four match registers, which are 32 bits in length and can be loaded with numbers.
- One of them can be used for a timer operation.
- When the timer count register increments to become equal to the number in the match register, a match is said to have occurred.
- MATCH CONTROL REGISTERS (MCR): The bit configuration of the Match Control Register determines the course of action following a 'match'.

Registers for timing operation

There are 4 match register .For each three bits of the MCR are used shown in fig 6.13 The next three bits of MCR are allocated for MR1 and so on.



Figure 6.13 Important bits of Match Control Register of MR0

For MR0, the bits of MCR are

Bit 0: When this bit is made '1' an interrupt is to be generated on match.

Bit 1: When this bit is made '1', the Timer count register is to be reset on match.

Bit 2: When this bit is made '1', the Timer count register is to be stopped on match. The prescale counter is also stopped, and the enable bit of the Timer control register (TCR) is made 0, so that the counting is disabled.

Generating Square wave

Now let us use the above information to generate a square wave using Timer0. We use the timer only for creating a delay. After each delay period is over, the pins of Port 0 are toggled. Thus, we get a square wave at all pins of the port. We use match register MR1.

```
-----  
#include <LPC214x.h>  
void wait(void);  
int main(void)  
{  
    T0MR1 = 0x000034FF;           // Load a number in MR1  
    T0MCR = 0x30;                 /*Stop and reset T0TC when match occurs*/  
    while(1)  
    {  
        IODIR0 = 0xFFFFFFFF;     //Make all pins of Port 0  
as output  
        IOPIN0=~IOPIN0;           // Toggle all pins of Port 0  
        wait();                   //Call the wait function  
    }  
}  
void wait(void)                   //The wait function  
{  
    T0TCR = 1;                    //Start the timer  
    while(!(T0TC==T0MR1));        // wait until T0TC = MR1  
}
```

Generating Square wave steps

Let us try to understand what has been done in the code.

1. The initial settings for the timer are:

(a) PCLK is obtained as $CCLK/4$ by settings in the VPB divider (this is not shown in the code).

(b) A number is loaded into Match Register MR1.

(c) To stop the Timer Counter (TC), and reset it on 'match', the corresponding bits in T0MCR is set to 1. These are bits no 4 and 5.
 $MCR = 0011\ 0000 = 0x30$.

2. To get a symmetrical square wave on all the pins of Port 0

(a) Port 0 is made an output port.

(b) An instruction to toggle all the port pins is used. IOPIN0 register is used as a whole.

(c) To specify the rate at which the toggling is to occur, a wait function is called.

3. The wait function is composed of the following actions

(a) The timer is started by setting the Enable bit of the Timer Control register ($T0CR=1$).

(b) The timer register updates its count, until it is equal to the value in T0MR1.

Frequency Calculation

- For calculation of the time period, we need a reference clock signal CCLK.
- Let use CCLK of 50 MHz it is divided by 4, to get a PCLK of 12.5 MHz.
- This translates to a PCLK period of $T = 0.08 \mu\text{s}$.
- The wait function creates a delay. The amount of delay is dependent on the number loaded in the Match register
- Load a value of 0x34FF (13567) in ToMR1
- The timer count register starts from 0 and counts up to 13,567 when match occurs.
- Thus, $13567+1 = 13568$ cycles of PCLK have to pass before match occurs.
- Thus, the delay obtained by this value in ToMR1 is $13567 \times 0.08 = 1085 \mu\text{s} = 1.085 \text{ ms}$.

Frequency Calculation contd...

- During this time, all the pins of Port 0 have one logic level. Then the pin toggles to the alternate logic level.
- Thus, the time period of the square waveform T , is $1.085 \times 2 = 2.170 \text{ ms}$.
- The frequency of the waveform is the reciprocal of the timer $= 1/T = 0.460 \text{ KHz} = 460 \text{ Hz}$

Vectored Interrupt Controller

- The Vectored Interrupt Controller (VIC) takes 32 interrupt request inputs and programmably assigns them into three categories – FIQ, Vectored IRQ and non-vectored IRQ.
- FIQ requests have the highest priority. If more than one request is assigned to FIQ, the VIC does the ORing of the requests to produce FIQ signal to the processor.
- Vector IRQ have the middle priority, but only 16 of the 32 requests can be assigned to this category. Any of the 32 requests can be assigned to any of the 16 vectored IRQ slots, among which slot 0 has the highest priority and slot 15 has lowest priority.
- Non-vectored IRQ have the lowest priority
- The VIC performs ORing of the requests from all the vectored and non vectored IRQ's to produce the IRQ signal to the ARM processor.

Interrupt Sources

Table 6.5 Channel numbers for interrupt sources

IRQ	Flag(s)	VIC Channel # Mask (Bits)
WDT	Watchdog Interrupt (WDINT)	0
NA	Reserved	1
ARM Core	Embedded ICE, DbgCommRx	2
ARM Core	Embedded ICE, DbgCommTx	3
TIMER0	Match0-3 (MR0, MR1, MR2, MR3) Capture0-3 (CR0, CR1, CR2, CR3)	4
TIMER1	Match0-3 (MR0, MR1, MR2, MR3) Capture0-3 (CR0, CR1, CR2, CR3)	5
UART0	UART 0 Interrupt	6
UART1	UART 1 Interrupt	7
PWM0	Match0-6 (MR0, MR1, MR2.....MR6)	8
I2C0	SI (State Change)	9
SPI0	SPI Interrupt Flag (SPIF)	10
SPI(SSP)	SPI Interrupt Flag	11
PLL	PLL Lock (PLOCK)	12
RTC	Counter Increment (RTCIF)	13
System Control	External Interrupt 0 (EINT0)	14
	External Interrupt 1 (EINT1)	15
	External Interrupt 2 (EINT2)	16
	External Interrupt 3 (EINT3)	17
ADC0	A/D Converter 0 end of conversion	18
I2C1	SI (State Change)	19
BOD	Brown Out Detect	20
ADC1	A/D Converter 1 end of conversion	21
USB	USb Interrupt, DMA Interrupt	22

Important registers of VIC

Interrupt Enable Register (VIC Interrupt Enable)

- This is a read/write accessible register.
- This register controls the decision of which of the 32 interrupt requests.
- Example:
 - VIC Int Enable = 0x00000010; //Enable the TIMER-0 interrupt by making bit 4=1

Bit Definitions of VIC

Table 6.6 Bit definitions of VIC IE register for some of the peripherals of the chip

Bit	7	6	5	4	3	2	1	0
Symbol	UART 1	UART 0	TIMER 1	TIMER 0	ARM Core 1	ARM Core 0	—	WDT

Table 6.7 The bits of the VIC Vector Control register for enabling the Timer 0 interrupt

Bits	Function	For Timer 0
4:0	The number of the selected interrupt	00100
5	Enable the chosen IRQ slot	1

Vector control register (VIC Vect CntIO-15)

Vector control register (VIC Vect CntIO-15)

- Only 6 bits of this register are to be used.
- They are lower 6 ones
- In Table 6.7, Timer 0 is allotted the number 4.
- Table 6.6 illustrates how this register is to be configured
- Thus, Example 6.7 uses the code line `VICVectCnt14 = 0x00000024`

Vector address register (VIC VectAdd)

Vector address register (VIC VectAdd):

- These are read/write accessible registers.
- These registers hold the addresses of the interrupt service routines (ISRs) for the vectored IRQ slots.
- `VICVectAddr4 = (unsigned) Timeroisr; //set the timer ISR vector address`

Timer 0 interrupt register (ToIR)

Timero interrupt register (ToIR)

- This register has bits for each of the matching states of MR0 to MR3.
- When a timer operates in the interrupt mode and a match occurs, an interrupt is generated, and the corresponding flag bit in ToIR is set.
- To 'clear' it, a '1' must be written into this same register.
- Then only will the interrupt flag be 'reset'
- To1R = 0x01

Timer 0 interrupt register (ToIR)

Bit	Symbol	Description
0	MR0 Interrupt	Interrupt flag for match channel 0.
1	MR1 Interrupt	Interrupt flag for match channel 1.
2	MR2 Interrupt	Interrupt flag for match channel 2.
3	MR3 Interrupt	Interrupt flag for match channel 3.

Pulse Width Modulator

Pulse Width Modulator:

- All advanced MCUs have PWM as a separate unit.
- In LPC214x, if the PWM mode is not enabled, the unit behaves as a timer.
- There is a facility to obtain 6 PWM waveforms simultaneously from MCU.
- There are specific pins assigned as the output pins.
- Pin select registers are to be used for enabling the pins to work as PWM output pins.

Pulse Width Modulator

Table 6.9 PWM output pins of the six PWM channels

PWM no	Channel no	Output Pin no
PWM1	1	P0.0
PWM2	2	P0.7
PWM3	3	P0.1
PWM4	4	P0.8
PWM5	5	P0.21
PWM6	6	P0.9

Pulse Width Modulator contd...

Find the appropriate PINSEL registers and the values to be used to select PWM6, PWM5 and PWM4.

	Port Pins	Pin Select register	Bits of Pin Select register	Bit values
PWM4	P0.8	PINSEL0	17:16	10
PWM5	P0.21	PINSEL1	11:10	01
PWM6	P0.9	PINSEL0	19:18	10

As per the above table, the register

PINSEL0 = 0000 0000 0000 1010 0000 0000 0000 0000 = 0x000A0000

PINSEL1 = 0000 0000 0000 0000 0000 0100 0000 0000 = 0x00000400

.....

Pulse Width Modulator

Functions of PWM:

- Concept of match registers are used here like timer. There is a match register MR₀.
- The number loaded into MR₀ determines the pulse repetition rate of PWM waveform.
- MR₀ determines the time period T of the waveform. There are 6 other match registers MR₁ to MR₆.
- They are used for deciding ON period of each of 6 PWM signals PWM₁ to PWM₆.
- Fig 6.16 shows a pulse stream in which t is the ON period and T, the pulse period. The duty cycle is defined as

$$\text{Duty cycle} = t/T \times 100 \%$$

If six PWM outputs are used in the single edge controlled scheme, all of them go high at the same time, but the fall time of the pulse are different and determined by the values in registers MR₁ to MR₆.

How does the PWM unit function?

Fig 6.17 shows the case of three PWM outputs generated using the single edge controlled scheme. MR₁ to MR₃ are used for determining the ON times of each of the pulse trains (see table 6.10)

Table 6.10 PWM channels and corresponding match registers for the single edge controlled scheme

PWM Channel	T set by (T)	On time by (t)
1	Match 0	Match 1
2	Match 0	Match 2
3	Match 0	Match 3
4	Match 0	Match 4
5	Match 0	Match 5
6	Match 0	Match 6

PWM waveform

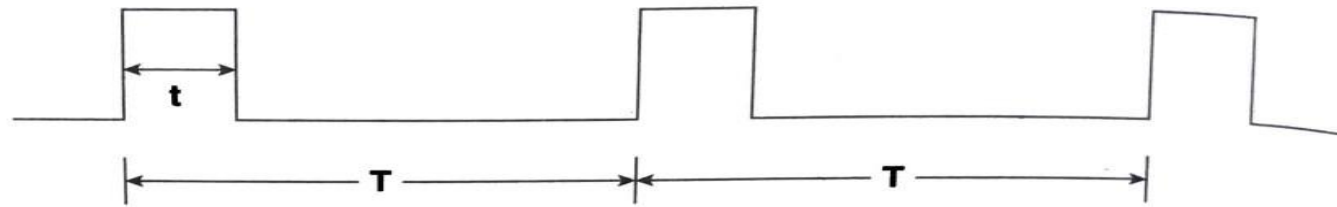


Figure 6.16 A PWM waveform

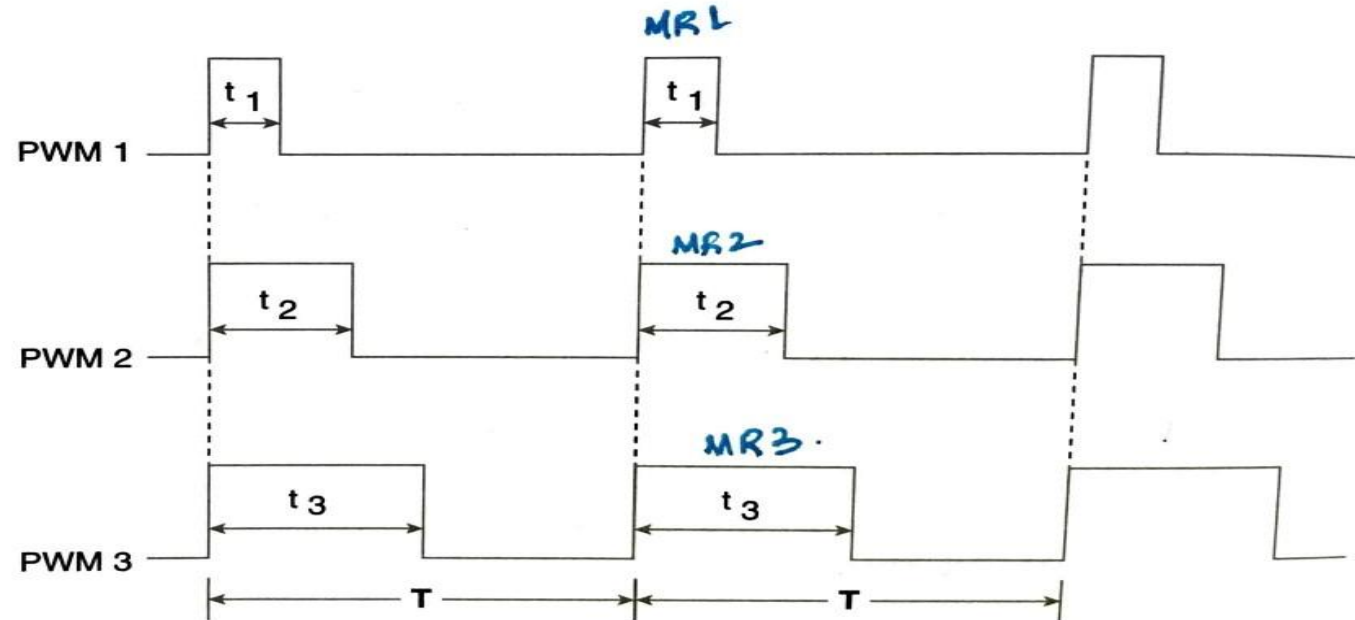


Figure 6.17 PWM waveforms using the single edge controlled scheme

PWM waveform

EXAMPLE 6.9

Load PWMMR0 with the number 0x 4556. Load PWMMR1 and PWMMR2 with the values 0x2341 and 0x0E40. Find the duty cycles of the PWM outputs PWM1 and PWM2.

SOLUTION

Let us take PCLK to be one fourth of CCLK which is 50 MHz. PCLK is thus 12.5 MHz, and the time period of PCLK is $0.08 \mu\text{s}$. When MR0 = 0x4556, this number determines the pulse period T. 0x4556 in decimal is 17,750.

To count from 0 to 17,750, it takes 17,751 cycles of PCLK.

$1420.8 \mu\text{s} = 1.42 \text{ ms}$. This is the time period T.

For calculating the ON time:

MR1 = 0x2341. 0x2341 in decimal is 9025

The number of cycles of PCLK is 9026.

ON period of PWM1 = $9026 \times 0.08 = 722 \mu\text{s}$

With MR2 = 0x E40, the equivalent decimal value is 3648

The number of cycles of PCLK is 3649.

ON period of PWM2 = $292 \mu\text{s} = 0.292 \text{ ms}$

The calculation of the duty cycle of the two pulse trains is given in Table 6.11

PWM waveform

Table 6.11 Duty cycle calculation of Example 6.9

Name of PWM output	t	T	Duty cycle = t/T
PWM1	0.722 ms	1.42 ms	50.8%
PWM 2	0.292 ms	1.42 ms	20.56%

Find the numbers to be loaded in MR3 and MR4 to get duty cycles of 20% and 65% at pins PWM3 and PWM4, when $T = 7.5$ ms.

EXAMPLE 6.10

For a period T of 7.5ms, the number to be loaded in MR0 is to be found out in this way:

SOLUTION

$$7500 \mu s = 0.08 \times (N+1)$$

$$N = 93749 = 0x12015$$

This is the hexadecimal number to be loaded into MR0.

To find the number ($N1$) to be loaded in MR3 –

The ON time is 20% of 7.5 ms = 1.5 ms

$$1500 \mu s = 0.08(N1+1)$$

$$N1 = 18749 = 0x493D$$

Registers of PWM unit

- The PWM unit is a timer, which has a Timer Count register.
- The value of which is compared to the various match registers, and actions are initiated accordingly.
- When using the PWM unit, we do not load anything in this Timer Count Register, instead, all operations are done by the use of the match registers

PWM Control Register (PWMPCR)

- The PWM Control Register is used to enable and select the type of each PWM channel.
- This register enables or disables the six PWM outputs, and also choose between double and single edge control. Bits 0, 1, 7 and 8-15 are unused.
- Table 6.13 shows the state of the bits of PWMPCR for choosing between single and double edge control.

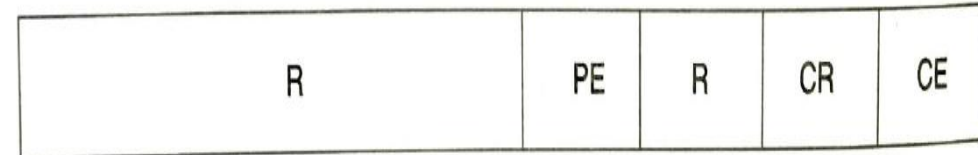
PWM Control Register (PWMPCR) contd...

Table 6.13 Bits of the PWMPCR

Bit No of PWMPCR	When Bit = 0	When Bit = 1
PWMPCR.2	Single edge control for PWM2	Double edge control for PWM2
PWMPCR.3	Single edge control for PWM3	Double edge control for PWM3
PWMPCR.4	Single edge control for PWM4	Double edge control for PWM4
PWMPCR.5	Single edge control for PWM5	Double edge control for PWM5
PWMPCR.6	Single edge control for PWM6	Double edge control for PWM6

Figure 6.18

Important bits of PWMTCR



PWM Timer Control Register (PWMTCR)

The functions of each of the bits is shown in fig 6.18.

- **Bit 0- counter enable:** when set, the PWM Timer counter and PWM prescale counter are enabled for counting. When 0, counters are disabled.
- **Bit 1- Counter Reset:** When set, the PWM timer count register and the PWM prescale counter are synchronously reset on the next positive edge of PCLK. Counters remain reset until this bit is made 0.
- **Bit 2:** reserved.
- **Bit 3- PWM Enable:** when set, PWM is enabled, if 0, PWM unit acts as a timer.
- **Bit 4-15:** Reserved



Figure 6.18
Important bits of
PWMTCR

The PWM match register that determines the PWM rate(MRo) must be set prior to the PWM being enabled.

PWM Latch Enable Register (PWMLER)

- **PWM Latch Enable Register (PWMLER)**
- It is an 8-bit register used to control the update of the PWM match registers when they are used for PWM generation.
- When software writes to the location of a PWM match register while the timer is in PWM mode, the value is held in a shadow register.
- When a PWM Match 0 event occurs, the contents of shadow registers will be transferred to the actual match registers if the corresponding bit in the latch enable register has been set.
- At that point, the new values will take effect and determine the course of the next PWM cycle. Once the transfer of new corresponding bit in the PWMLER is set and a PWM Match 0 event occurs.

PWM match Registers

- PWM Match Registers - PWMMR0 – PWMMR6
- The 32-bit PWM Match register values are continuously compared to the PWM Timer Counter value.
- When the two values are equal, actions can be triggered automatically.
- The action possibilities are to generate an interrupt reset the PWM Timer Counter, or stop the timer.

PWM prescale register (PWMPR)

- The 32-bit PWM Prescale Register specifies the number by which the PCLK is slowed down, similar to the case of the Timer.
- There are many other registers for PWM, but here we will generate PWM output signals with these few registers.

Universal asynchronous receiver/trans mitter

- A UART is a mandatory unit in every MCU.
- In LPC 214x chip, there are two UARTs in the chip – UART₀ and UART₁.
- The UART is a unit which does serial transmission on the pin TxD and serial reception on the pin RxD.
- For the chip there are two transmit and receive pins each.

Serial data transmission and reception

The mechanism employed in serial data transmission and reception:

- Inside the MCU, all data communication is done in parallel.
- Data is moved around as either 32, 16 or 8 bits.
- Thus, when data is to be transmitted or received serially, there is a parallel to serial (and vice versa) conversion involved.
- For transmission, the data from a buffer register is converted to serial form, framed by adding the start, stop and parity bits and sent on the TxD pin.
- In reception, the bits received are kept in a buffer (named the hold register), de-framed, parallelized and moved as one byte to the CPU.
- Reception and transmission are done at a rate commonly known as the baud rate.
- There is a baud rate generator inside.
- Baud rate can be decided by a calculation depending on the value of certain numbers in the UART registers.
- The UART can work in the polled mode or in the interrupt mode.

Serial data transmission and reception contd

- Figure below shows the conceptual view of the UART operation.
- In transmission, there is a Transmit FIFO in which the bytes to be transmitted are stored.
- The 8-bit register Transmit Hold Register(THR) is the top byte of the UART transmit FIFO.
- The top byte is the newest character in the Transmit FIFO and can be written via the bus interface.
- The LSB represents the first bit to be transmitted.
- From THR, the data is framed and serialized.
- The TSR (Transmit Shift Register) does the serialization of data, and sends it out, bit by bit on the TxD pin.

Serial data transmission and reception contd

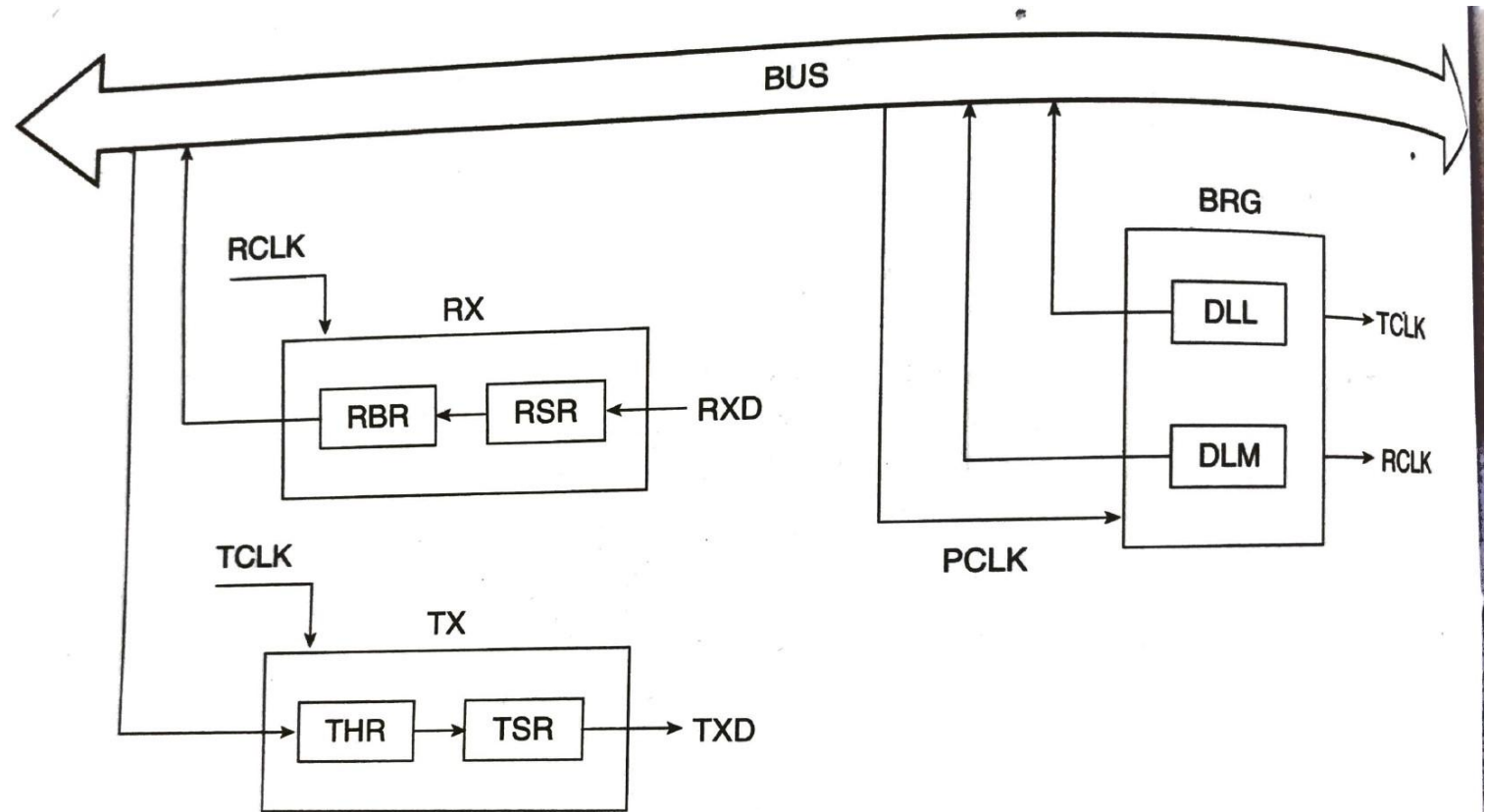


Figure 6.22 Transmission and reception mechanism in the UART

Serial data transmission and reception contd

- Reception is similar.
- The received serial data comes in the Receiver Shift Register (RSR)
- It is deframed, parallelized and saved in the receiver FIFO.
- The Receiver Buffer Register (RBR) is the top byte of the UART receiver FIFO
- The top byte of the Receive FIFO contains the oldest character received and can be read via the bus interface.
- The LSB represents the oldest received bit.
- If the character received is less than 8 bits, the unused MSBs are padded with zeroes.

The BAUD Rate Generator

- Transmission and reception rates are referred to as TCLK and RCLK.
- These clocks are generated by the BRG (Baud Rate Generator) and these rates can be fixed by a formula.

Registers of UART₀

UoDLL and UoDLM – Divisor Latch registers:

- Both of them hold 8-bit values.
- These register together form a 16-bit divisor value which is used in baud rate generation.
- UoDML holds the upper 8-bits and UoDLL holds the lower 8-bits
- Since these form a divisor value and division by zero is invalid, the starting value for UoDLL is 0x01 (and not 0x00), i.e. the starting value in combined formation is 0x0001.
- In order to access and use these registers properly, DLAB bit in UoLCR must be first set to 1.
- The factor $MULVAL / (MULVAL + DIVADDVAL)$ depends on the prescaler and another register.

Registers of UAERo contd

The formula for baud rate calculation is

$$UART0_{baudrate} = \frac{PCLK}{16 \times (16 \times U0DLM + U0DLL)} \times \frac{MulVal}{(MulVal + DivAddVal)}$$

Using this formula ,for a system with PCLK=20 MHz, UoDLM=0x00 and UoDLL=0x5D ,DIVADDVAL=2 and MULVAL=5.
The UART baud rate=9600bauds.

UARTo transmit holding register (UoTHR)

- This is an 8-bit register and part of the transmit buffer.
- It is the top most byte of this buffer, and new characters are to be loaded into this register for being transmitted.
- The data to be transmitted is written into this 'write only' register.

UARTo FIFO control registers(UoFCR)

- This is a 8-bit register
 - Bit 0: E: This bit must be set for enabling the Tx and Rx FIFOs
 - Bit 1: Rx FIFO Reset: This must be set, to clear all bytes in UARTo RX FIFO and reset the pointer logic. This bit is self-clearing.
 - Bit 2: Tx FIFO Reset: This must be set to clear all bytes in UARTo Tx FIFO and reset the pointer logic. This bit is self-clearing.
 - Bit 7 and 6: Rx trigger level: these two bits determine how many receiver FIFO characters must be written before an interrupt is activated

Figure 6.23 Bits of
UoFCR

T (Bit 7 and 6)	Reserved (Bits 3 to 5)	T × R (Bit 2)	R × R (Bit 1)	E (Bit 0)
-----------------	------------------------	---------------	---------------	-----------

UARTo line control register (UoLCR)

- The UoLCR is an 8-bit register which determines the format of the data character that is to be transmitted or received.
- The bits used here are:
 - Bit 1: 0: These two bits have been chosen to be '11' to indicate 8-bit character length.
 - Bit 2: This is made '0' to select one stop bit.
 - Bit 7: This is the Divisor Latch Access Bit (DLAB) and is set, to enable the use of divisor latch.
- Other bits of this register pertain to parity and break control.
- These have been disabled by making these bits to be '0'.
- Thus we have $\text{UoLCR} = 1000\ 0011 = 0x83$

UARTo line status register (UoLSR)

- The UoLSR is a read-only register that provides status information regarding the UARTo Transmitter and Receiver blocks.

UARTo pins in the PINSELo register

Pinselect register (PINSELo)

This register has been discussed earlier. Po.o(TxD) and Po.1(RxD) are the relevant pins of UARTo, PINSELo=oxo5 is to be used. See table 6.14

Table 6.14 The UART0 pins in the PINSELo register

Bits	Port Pin	Value	Function
1:0	P0.0	00	GPIO
		01	TxD (UART0)
		10	PWM1
		11	Reserved
3:2	P0.1	00	GPIO
		01	RxD (UART0)
		10	PWM3
		11	EINT0

The program simulates serial communication. A character string written in the program is transmitted, received and is displayed in the serial window.

EXAMPLE 6.13

```
#include<LPC214X.h>
void init()
{
    PINSEL0=0x05; // select TxD and RxD pins of UART0
    U0FCR=0x07;   //Enable and clear FIFOs
    U0LCR=0x83;   //Enable divisors
    U0DLL=0x5D;   // 9600 baud
    U0DLM=0x00;   // 9600 baud
}
void delay()
{
    int i;
    for(i=0;i<250;i++);
}
int main()
{
    unsigned char p[]="I LOVE EMBEDDED SYSTEMS\n";
    int z;
    init();
    for(z=0;z<=24;z++)
    {
        U0THR=p[z];
        while(!(U0LSR&0x20)); // check status of bit of U0LSR
        delay();
    }
    while(1);
}
```